

DATA STRUCTURE - II

UNIT - I

Tree: Header nodes, Threads, Binary search trees, Searching, Insertion and deletion in a Binary search tree, AVL search trees, Insertion and deletion in AVL search tree, m-way search tree, Searching, Insertion and deletion in an m-way search tree, B-trees, Searching, Insertion and deletion in a B-tree, B+tree, Huffman's algorithm, General trees.

TREE:-

- A tree is a nonlinear data structure that represents data in a hierarchical manner. It associates every object to a node in the tree and maintains the parent/child relationships between those nodes.
- A graph which has no cycle is called an acyclic graph. A tree is an acyclic graph.
- A tree can be empty with no nodes called the null or empty tree.

Tree Data Structure Terminology-

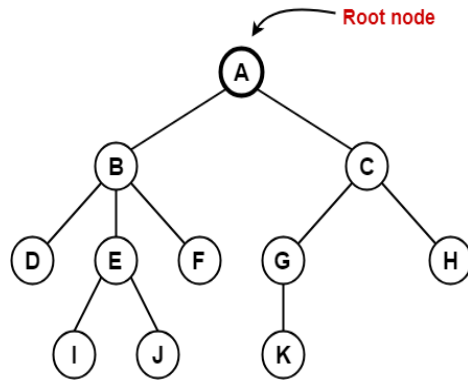
Following are few important terms related to tree data structure:-

1. Root
2. Edge
3. Parent
4. Child
5. Siblings
6. Degree
7. Internal node
8. Leaf node
9. Level
10. Height
11. Depth
12. Subtree
13. Forest

1. Root-

- Root node is the origin of tree data structure. It is the first node.
- In any tree, there must be only one root node. We can never have multiple root nodes in a tree data structure.

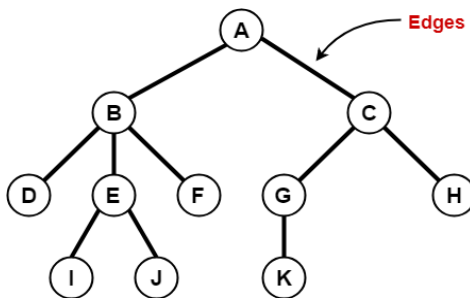
Example-



2. Edge-

- The connecting link between any two nodes is called as an edge.
- In a tree with n number of nodes, there are exactly n-1 numbers of edges.

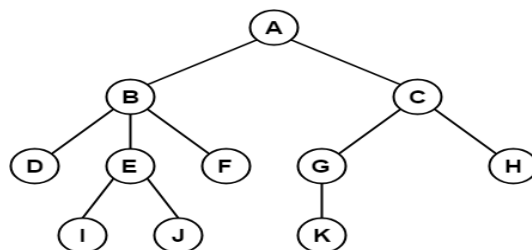
Example-



3. Parent-

- The node which has a branch from it to any other node is called as a parent node.
- In other words, the node which has a child / children is called as a parent node.

Example-

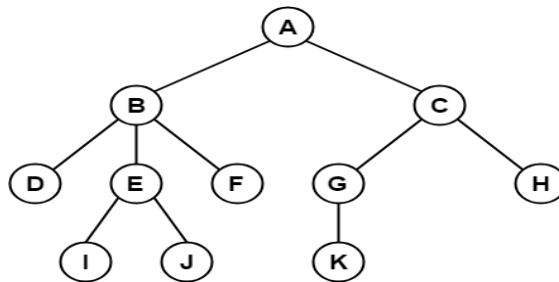


Here, A, B, C, E and G are parent nodes.

4. Child-

- The node which is descendant of any node is called as a child node.
- All the nodes except root node are child nodes.
- In a tree, any parent node can have any number of child nodes.

Example-



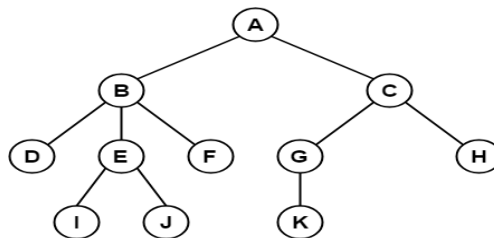
Here,

- B and C are children of A
- G and H are children of C
- K is the only child of G etc.

5. Siblings-

- Nodes which belong to the same parent are called as siblings.
- In other words, nodes with the same parent are called as sibling nodes.

Example-



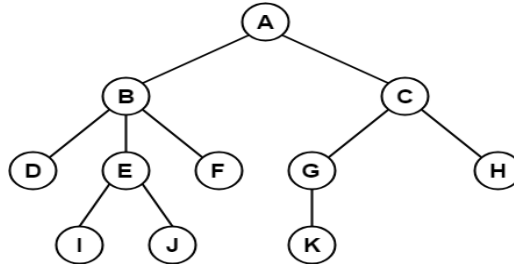
Here,

- B and C are siblings
- D, E and F are siblings
- G and H are siblings
- I and J are siblings

6. Degree-

- The total number of children associated with a node is called as degree of that node.
- Degree of the tree = Highest degree of a node among all the nodes in the tree

Example-



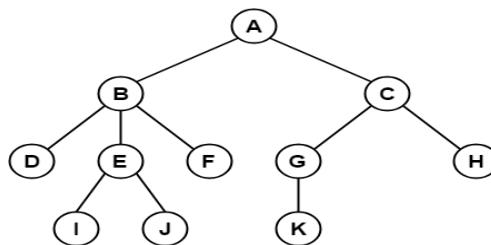
Here,

- Degree of node B = 3
- Degree of node A = 2
- Degree of node F = 0 etc

7. Internal node-

- The node which has at least one child is called as an internal node.
- Internal nodes are also called as non-terminal nodes.
- Every non-leaf node is an internal node.

Example-

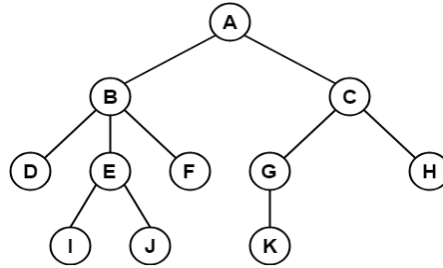


Here, A, B, C, E and G are internal nodes.

8. Leaf node-

- The node which does not have any child is called as a leaf node.
- Leaf nodes are also called as external nodes or terminal nodes.

Example-

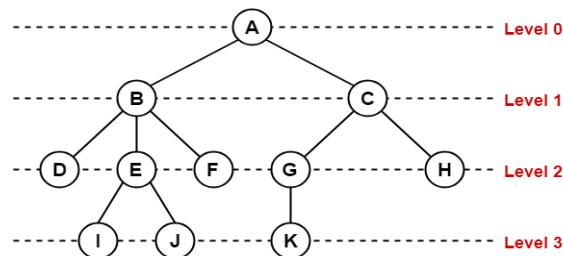


Here, D, I, J, F, K and H are leaf nodes.

9. Level-

- In a tree, each step from top to bottom is called as a level.
- The level count starts with 0 and increments by 1 at each level or step

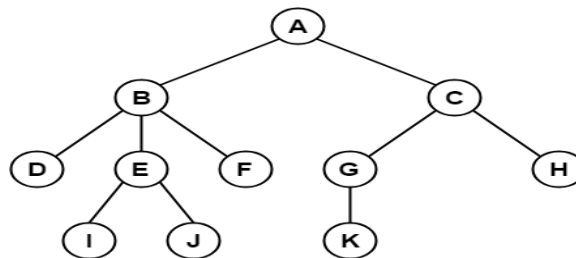
Example-



10. Height-

- The total number of edges that lies on the longest path from any leaf node to a particular node is called as height of that node.
- Height of the tree = Height of the root node
- Height of all leaf nodes = 0

Example-



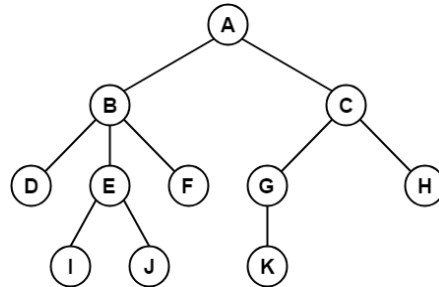
Here,

- Height of node K = 0
- Height of node B = 2
- Height of node A = 3
- Height of node G = 1
- Height of node H = 0 etc

11. Depth-

- The total number of edges from root node to a particular node is called as depth of that node.
- Depth of the tree = Total number of edges from root node to a leaf node in the longest path
- Depth of the root node = 0
- The terms “level” and “depth” are sometimes used interchangeably.

Example-



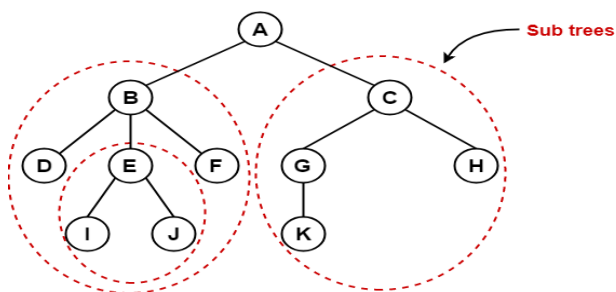
Here,

- Depth of node A = 0
- Depth of node B = 1
- Depth of node K = 3 etc

12. Subtree-

- In a tree, each child from a node forms a sub tree recursively.
- Every child node forms a sub tree on its parent node.

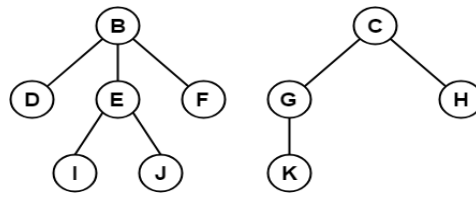
Example-



13. Forest-

- A forest is a set of disjoint trees.

Example-



Forest

BINARY TREES

Binary tree:- Binary tree is that tree in which every node can have 0,1 or 2 offspring's or child.

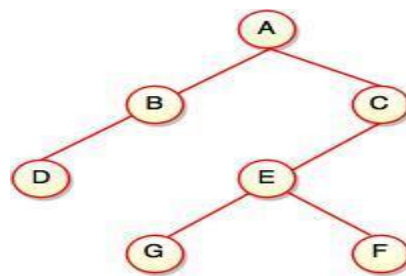
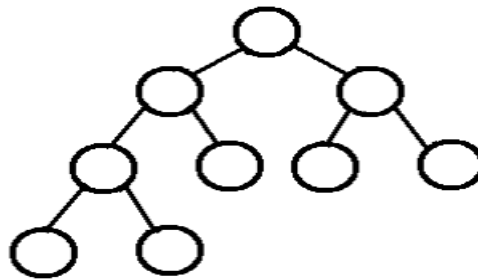


Fig. Binary Tree

Complete Binary tree:- A Binary tree T is said to be complete if all its levels, except possibly the last, have the maximum number of possible nodes, and if all the nodes at the last level appear as far left as possible.

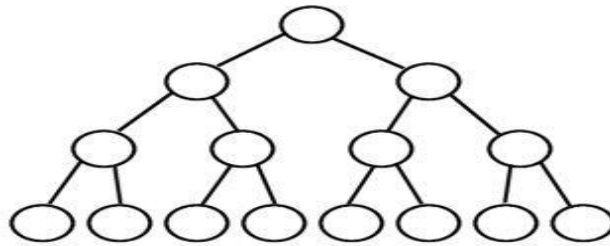


complete tree

Full Binary tree:-

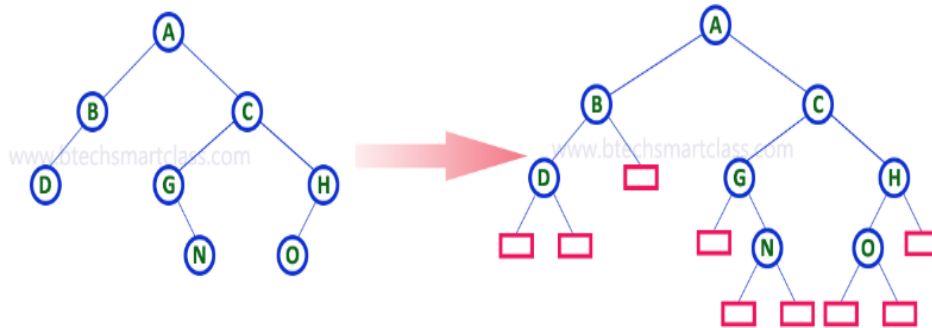
A binary tree is said to be a full binary tree if all the levels contains maximum possible node. Every node except the leaf node has exactly two child nodes.

Full Binary Tree



Extended Binary Tree:-

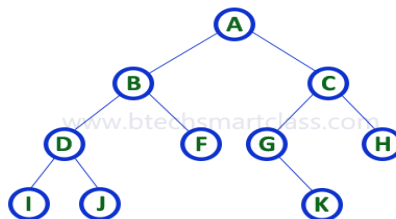
A binary tree can be converted into Full Binary tree by adding dummy nodes to existing nodes wherever required.



Representation of Binary Tree:-

1. Array Representation
2. Linked List Representation

Example:-



1. Array Representation of Binary Tree:-

In array representation of binary tree, we use one dimensional array (1-D Array) to represent a binary tree.

Array representation of the above Binary tree is shown as follows:-

A	B	C	D	F	G	H	I	J	-	-	-	K	-	-	-	-	-	-	-
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

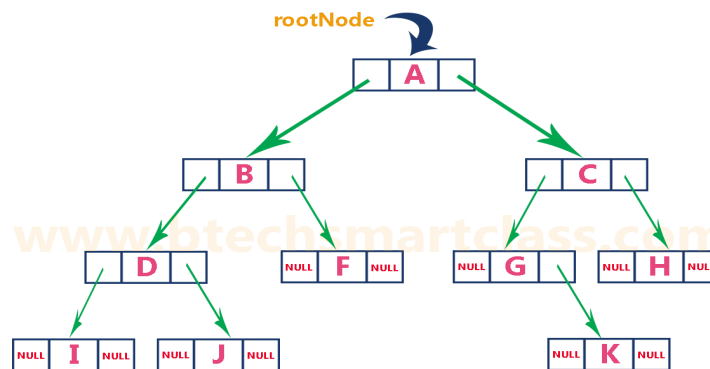
To represent a binary tree of depth ' n ' using array representation, we need one dimensional array with a maximum size of $2n + 1$.

2. Linked List Representation of Binary Tree:-

- We use double linked list to represent a binary tree. In a double linked list, every node consists of three fields.
- First field for storing left child address, second for storing actual data and third for storing right child address. In this linked list representation, a node has the following structure:-

Left Child Address	Data	Right Child Address
-----------------------	------	------------------------

- The above example of binary tree represented using Linked list representation is shown as follows:-



TREE TRAVERSAL ALGORITHMS:- Traversal of a binary tree means to visit each node in the tree exactly once. In a linear list, nodes are visited from first to last, but since trees are nonlinear we need to definite rules to visit the tree. There are a number of ways to traverse a tree. All of them differ only in the order in which they visit the nodes.

The three main methods of traversing a tree are:

- Preorder Traversal
- Inorder Traversal
- Postorder Traversal

Preorder Traversal:-

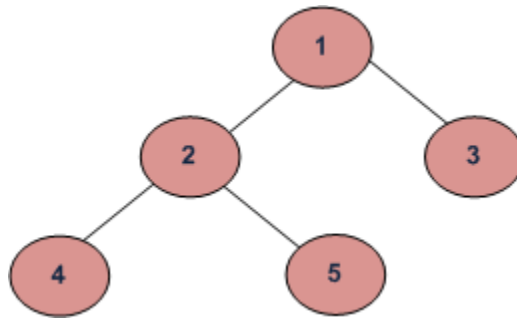
In this traversal process, it visits every node as it moves left until it can move no further. Now it turns right to begin again or if there is no node in the right, retracts until it can move right to continue its traversal.

1. Visit root node
2. Visit all the nodes in the left subtree
3. Visit all the nodes in the right subtree

Algorithm preorder (node)

```
if (node == null) then
    return
else
    visit (node)
    preorder (node->leftchild)
    preorder (node->rightchild)
```

Example:-



The preorder traversal is 12453

Inorder Traversal:-

The traversal keeps moving left in the binary tree until one can move no further, process node and moves to the right to continue its traversal again. In the absence of any node to the right, it retracts backward by a node and continues the traversal.

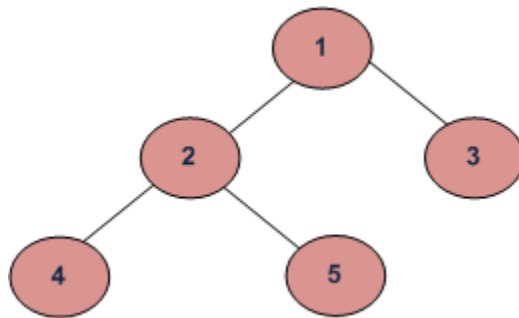
1. First, visit all the nodes in the left subtree
2. Then the root node
3. Visit all the nodes in the right subtree

Algorithm inorder(node)

```

if (node==null) then
    return
else
    inorder(node->leftchild)
    visit(node)
    inorder(node->rightchild)

```



42513 is the inorder traversal of the tree.

Postorder Traversal:-

The traversal proceeds by keeping to the left until it is no further possible, turns right to begin again or if there is no node to the right, processes the node and retraces its direction by one node to continue its traversal.

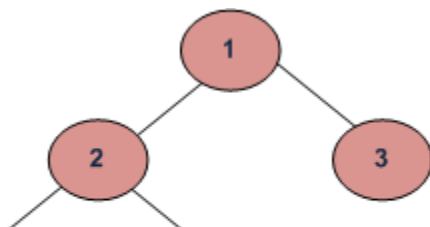
1. visit all the nodes in the left subtree
2. visit all the nodes in the right subtree
3. visit the root node

Algorithm postorder(node)

```

if (node==null) then
    return
else
    inorder (node->leftchild)
    inorder (node->rightchild)
    visit (node)

```



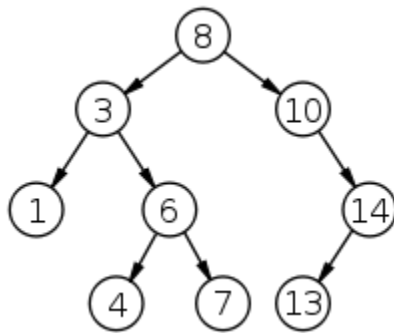
The postorder traversal of the tree is 45231

BINARY SEARCH TREE(BST): -

A binary tree is called binary search tree if

1. Each value in its left subtree less than the root value.
2. Each value in its right subtree greater than the root value.
3. Left and right subtrees are again binary search tree.

Example:-



The binary search tree is commonly represented using a linked representation in the same way as that of a binary tree. The binary search tree is also called a binary sorted tree.

Operations can we perform a BST include:

- Determine whether the binary search tree is empty.
- Search the binary search tree for a particular item.
- Insert an item in the binary search tree.
- Find the height of the binary search tree.
- Find the number of nodes in the binary search tree.
- Traverse the binary search tree.

Insertion in a Binary Search Tree:-

In a binary search tree, the insertion operation is performed with **$O(\log n)$** time complexity. In binary search tree, new node is always inserted as a leaf node. The insertion operation on a binary search tree is very simple. The insertion operation is performed as follows...

- **Step 1-** Create a newNode with given value and set its **left** and **right** to **NULL**.
- **Step 2-** Check whether tree is Empty.
- **Step 3-** If the tree is **Empty**, then set **root** to **newNode**.
- **Step4-** If the tree is **Not Empty**, then check whether the value of newNode is **smaller** or **larger** than the node (here it is root node).
- **Step5-** If newNode is **smaller** than or **equal** to the node then move to its **left** child. If newNode is **larger** than the node then move to its **right** child.
- **Step6-** Repeat the above steps until we reach to the **leaf** node (i.e., reaches to **NULL**).

- **Step7-** After reaching the leaf node, insert the newNode as **left child** if the newNode is **smaller or equal** to that leaf node or else insert it as **right child**.

Algorithm

Suppose the inserted node has data value VAL.

Step 1: Set (current node) + (root node).

Step 2: If (VAL = value stored at current node)

- Write duplicate value, insertion not possible
- Go to step 5.

Step 3: If (VAL < value stored at current node)

- If (left child exists)
 - Set (current node) $\square$ (left child node)
 - Go to step 2.
- If (left child do not exist)
 - Allocate a new node as left child node and insert value VAL in it.
 - Go to step 5.

Step 4: If (VAL > value stored at current node)

- If(right child exists)
 - Set (current node) $\square$ (right child node)
 - Go to step 2.
- If (right child do not exist)
 - Allocate a new node as right child node and insert value VAL in it.
 - Go to step 5.

Step 5: Stop.

C-Implementation

```
insert_node(structtreenode **ptr, intval)
{
    if ((*ptr) == NULL)
    {
        (*ptr) = (structtreenode)malloc(sizeoftreenode);
        (*ptr)->info = val;
        (*ptr)->lptr = NULL;
        (*ptr)->rptr = NULL;
    }
    else
    {
        if (val == (*ptr)->info)
            printf("Duplicate value, insertion not possible");
        else if (val < (*ptr)->info)
            insert_node(&((*ptr)->lptr), val);
        else
```

```

        insert_node(&((*ptr)->rptr), val);
    }
}

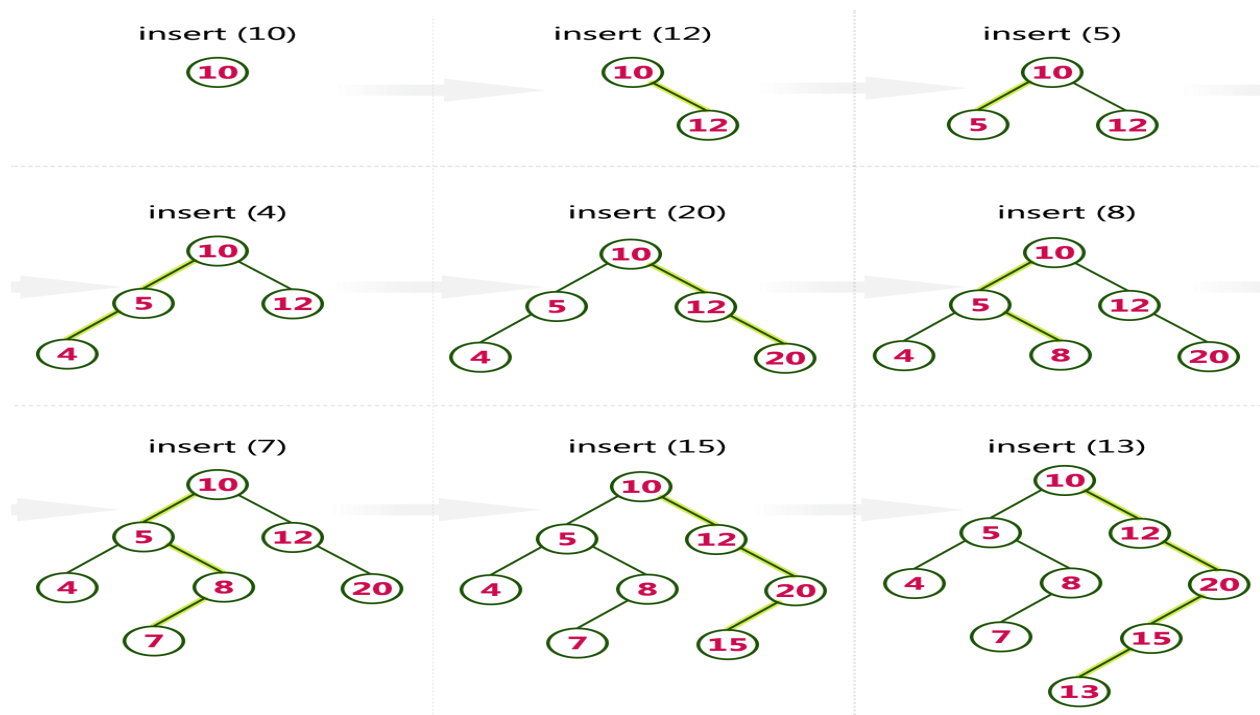
```

Example:-

Construct a Binary Search Tree by inserting the following sequence of numbers...

10,12,5,4,20,8,7,15 and 13

Above elements are inserted into a Binary Search Tree as follows...



Deletion Operation in BST: -

The deletion of a node in a binary search tree depends upon its specific position in the tree. Four possible cases to delete a node are:

- No node in the tree contains the specific data,
- The node containing the data is a leaf.
- The node containing the data has exactly one child.
- The node containing the data has two children.

In the first case when the specific data is not found, terminate the process.

In case of leaf node, it can be deleted by making its parent pointing to NULL. Now the deleted node is not referred and is disposed off.

If the node to be deleted has one child, then adjust the pointer of the parent of the deleted node to the child of that deleted node.

Algorithm:-

Step 1: (If the deleted node has no children) i.e.

If ((left(p) = NULL) and (right(p) = NULL)) the

- (i) Set parent(p) $\square$ NULL
- (ii) Free (p).
- (iii) Go to step 5.

Step 2: (If the deleted node has only one right child) in

If (left(p) = NULL) then

- (i) Set temp $\square$ p.
- (ii). Set p $\square$ right(p).
- (iii) Free (temp).
- (iv) Go to step 5.

Step 3: (If the deleted node has only one left child) i.e.

If (right(p) = NULL) then

- (i) Set temp $\square$ p.
- (ii) Set p $\square$ left(p).
- (iii) Free (temp).
- (iv) Go to step 5.

Step 4: (If the deleted node has 2 children) i.e.

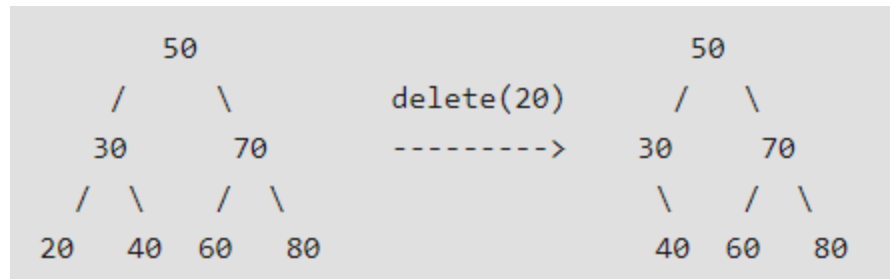
If ((left(p) $\neq$ NULL) and (right(p) $\neq$ NULL))

- (i) Set temp $\square$ right(p)
- (ii) While (left(temp) = NULL) repeat step
- (iii) Set temp $\square$ left(temp)
- (iv) Set left(temp) $\square$ left(p)
- (v) Set temp $\square$ p
- (vi) Set p $\square$ right(p)
- (vii) Free (temp)

Step 5: Stop.

When we delete a node, three possibilities arise.

1) Node to be deleted is leaf: Simply remove from the tree.



2) Node to be deleted has only one child: Copy the child to the node and delete the child.



3) Node to be deleted has two children: Find inorder successor of the node. Copy contents of the inorder successor to the node and delete the inorder successor. Note that inorder predecessor can also be used.



Search Operation in BST:-

- To compare an element in the binary search tree, first compare the target element T with the element at the root say R of the binary search tree.
- In a binary search tree, the search operation is performed with **O(log n)** time complexity. The search operation is performed as follows:-
- **Step 1** - Read the search element from the user.
- **Step 2** - If **(T=R)** i.e. the values are equal, search is successful and terminate the function.
- **Step 3** - If both are not matched **(T ≠ R)**, then
- **Step 4** - If **(T < R)** then by using the binary search tree property, search T in the left-child subtree. If there is no left-child subtree, the value is not found, i.e. search is unsuccessful. If there is a left-child subtree, then check it in the same way.

- **Step 5-** If ($T > R$) then by using the binary search tree property, search T in the right-child subtree. If there is no right-child subtree, the value is not found, i.e. search is unsuccessful. If there is a right-child subtree, then check it in the same way.

Algorithm:-

Suppose the searched element has data value VAL.

Step 1: Set (current node) $\square$ (root node).

Step 2: If ($VAL =$ value stored at current node)

- Write search successful.
- Go to step 5.

Step 3: If ($VAL <$ value stored at current node)

- If (left child exists)
 - Set (current node) $\square$ (left child node)
 - Go to step 2.
- If (left child do not exist)
 - Write there is no left subtree and search unsuccessful.
 - Go to step 5.

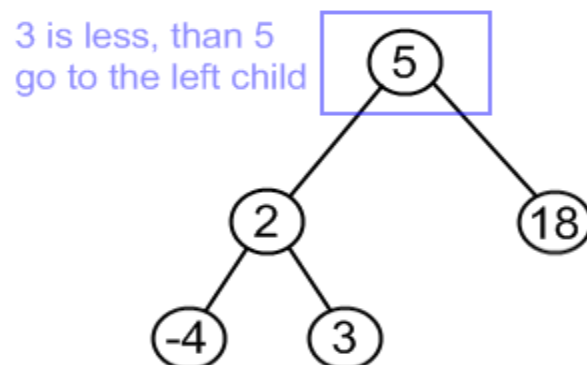
Step 4: If ($VAL >$ value stored at current node)

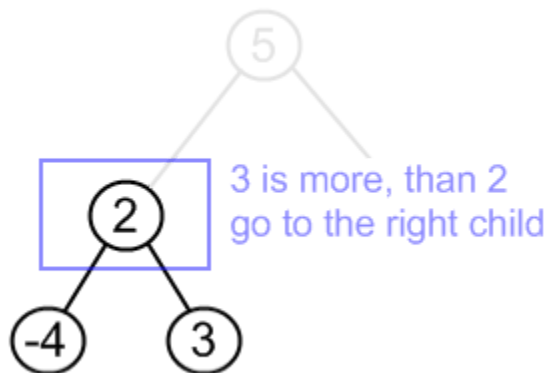
- If (right child exists)
 - Set (current node) $\square$ (right child node)
 - Go to step 2.
- If (right child do not exist)
 - Write there is no right subtree and search is unsuccessful
 - Go to step 5.

Step 5: Stop

Example

Search for 3 in the tree, shown above.





Threaded Binary Tree:-

Threads are pointer values used to replace null pointers and this process is called threading. Such a binary tree is called threaded binary tree. The objective of threading is to make effective use of null pointer.

- There are exactly $n+1$ null links for a tree of n nodes. This wasted memory space can be replaced by special links called threads which help in easy manipulations. The threads are shown by dotted lines.
- Binary trees have a lot of wasted space: the leaf nodes each have 2 null pointers.
- A binary tree is threaded according to a particular traversal order.

There are two types of threads. These are:-

1. Right thread:-

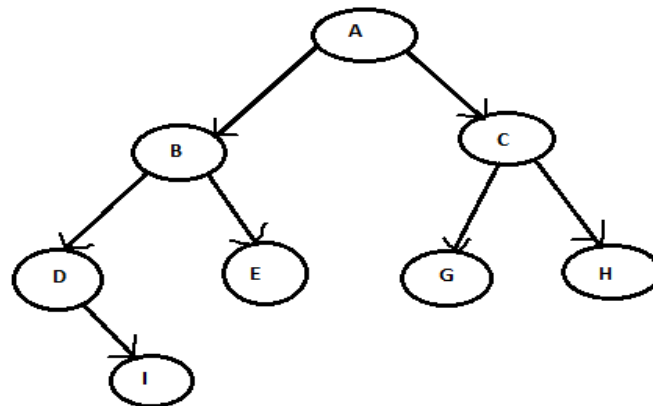
- A right thread links a node to its successor node in the inorder traversal.
- In a right threaded binary tree each NULL link is replaced by a special link to the successor of that node under inorder traversal called right threaded.

2. Left thread:-

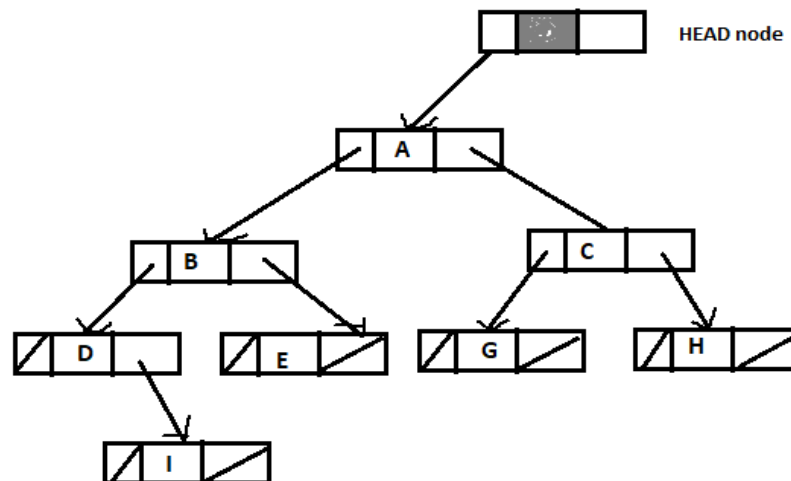
- A left thread links a node to its predecessor node in the inorder traversal.
- If one replaces each NULL left link by a special link to the predecessor of the node known as left threaded under inorder traversal the tree is known as left threaded binary tree.

If both the left and right threads are present in tree then it is known as fully threaded binary tree.

Example: - Consider a binary tree: -



Linked storage representation of the above binary tree:-



The HEAD node contains the address of root node.

Suppose we want to thread this binary tree according to inorder traversal.

Algorithm for inorder traversal order is:-

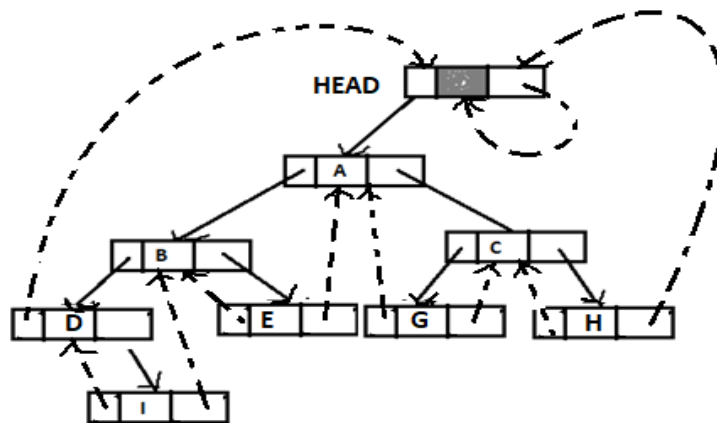
1. If the left link of node P is NULL, replace it by the address of predecessor of node P.

2. If the right link of node P is NULL, replace it by the address of successor of node P.
3. The HEAD node is simply another node which serves as the predecessor of first node and successor of the last node with respect to the inorder traversal.

The inorder of the above binary tree is:-

D I B E A G C H

Threaded representation for inorder traversal of this binary tree:-



AVL Tree:-

AVL tree is a binary search tree in which the difference of heights of left and right subtrees of any node is less than or equal to one.

An AVL tree can be defined as follows:

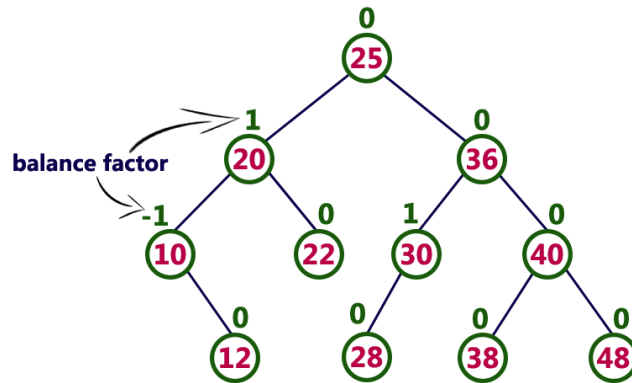
Let T be a non-empty binary tree with T_L and T_R as its left and right subtrees. The tree is height balanced if:

- T_L and T_R are height balanced
- $h_L - h_R \leq 1$, where h_L - h_R are the heights of T_L and T_R

Balance factor = height of (left subtree) h_L - height of (right subtree) h_R

The Balance factor of a node in a binary tree can have value 1, -1, 0, depending on whether the height of its left subtree is greater, less than or equal to the height of the right subtree.

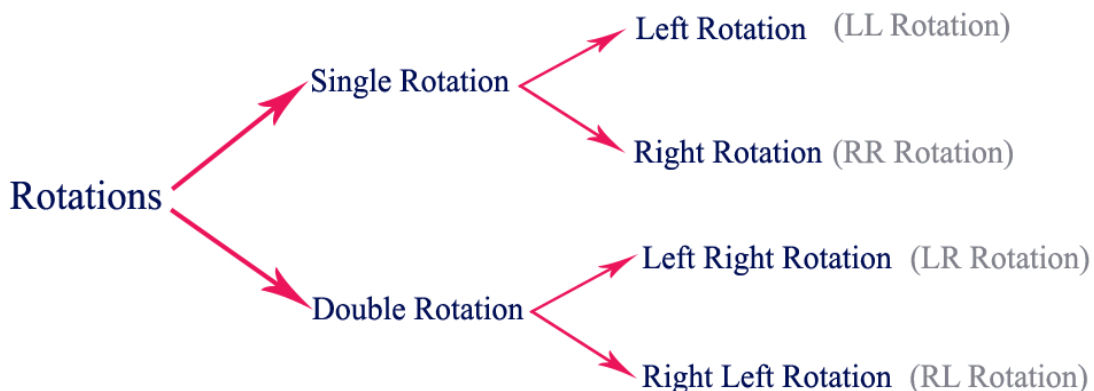
Example of AVL Tree:-



AVL Tree Rotations

In AVL tree, after performing operations like insertion and deletion we need to check the **balance factor** of every node in the tree. If every node satisfies the balance factor condition then we conclude the operation otherwise we must make it balanced.

- Whenever the tree becomes imbalanced due to any operation we use **rotation** operations to make the tree balanced.
- Rotation operations are used to make the tree balanced.
- There are four rotations and they are classified into two types.

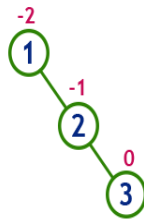


Single Left Rotation (LL Rotation)

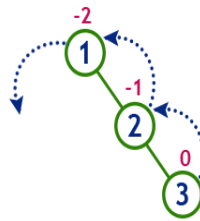
If a tree becomes unbalanced, when a node is inserted into the right subtree of the right subtree, then we perform a single left rotation: -

Example:-

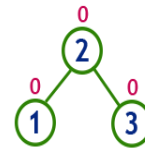
insert 1, 2 and 3



Tree is imbalanced



To make balanced we use LL Rotation which moves nodes one position to left



After LL Rotation
Tree is Balanced

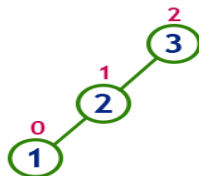
In the above example, node 1 has become unbalanced as a node is inserted in the right subtree of 1's right subtree. We perform the left rotation by making 1 the left-subtree of 2.

Single Right Rotation (RR Rotation)

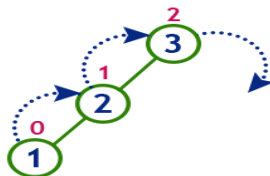
AVL tree may become unbalanced, if a node is inserted in the left subtree of the left subtree. The tree then needs a right rotation.

Example:-

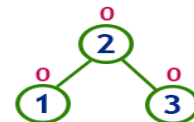
insert 3, 2 and 1



Tree is imbalanced
because node 3 has balance factor 2



To make balanced we use RR Rotation which moves nodes one position to right

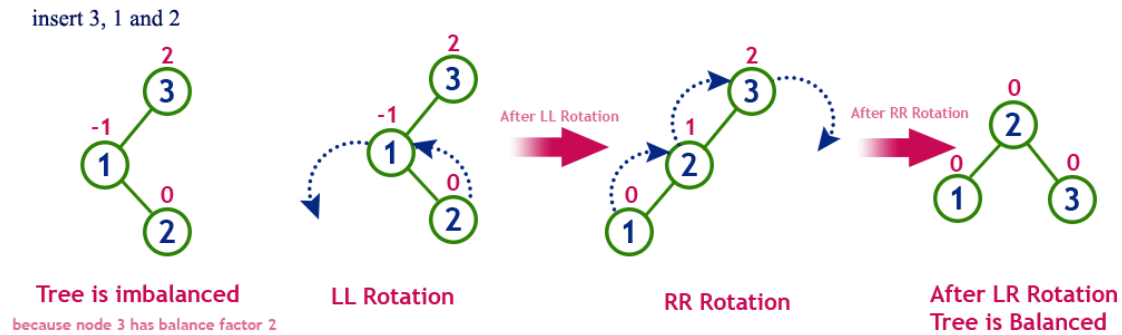


After RR Rotation
Tree is Balanced

Left Right Rotation (LR Rotation)

The LR Rotation is sequence of single left rotation followed by single right rotation. In LR Rotation, at first every node moves one position to left and one position to right from the current position.

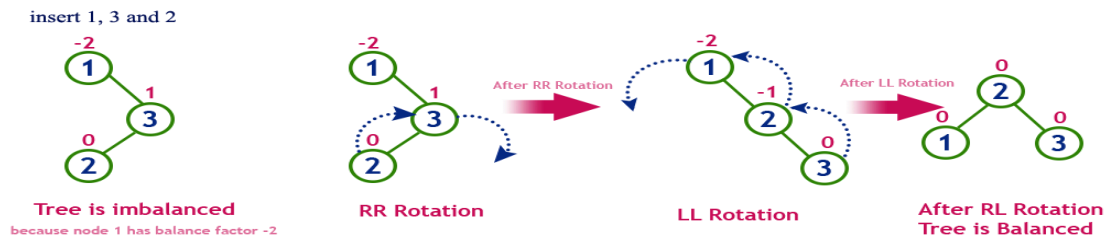
Example:-



Right Left Rotation (RL Rotation)

The RL Rotation is sequence of single right rotation followed by single left rotation. In RL Rotation, at first every node moves one position to right and one position to left from the current position.

Example:-



Insertion in AVL Tree:-

A new node is inserted into the balanced binary tree using the usual binary search tree insert procedure i.e. comparing the key of the new node with that in the root, and inserting the new node into the left or right sub-tree as appropriate.

Algorithm for Insertion:-

Step 1: First, insert a new element into the tree using BST's (Binary Search Tree) insertion logic.

Step 2: After inserting the elements you have to check the Balance Factor of each node.

Step 3: When the Balance Factor of every node will be found like 0 or 1 or -1 then the algorithm will proceed for the next operation.

Step 4: When the balance factor of any node comes other than the above three values then the tree is said to be imbalanced. Then perform the suitable Rotation to make it balanced and then the algorithm will proceed for the next operation.

Example: Construct an AVL Tree by inserting numbers from 1 to 8.

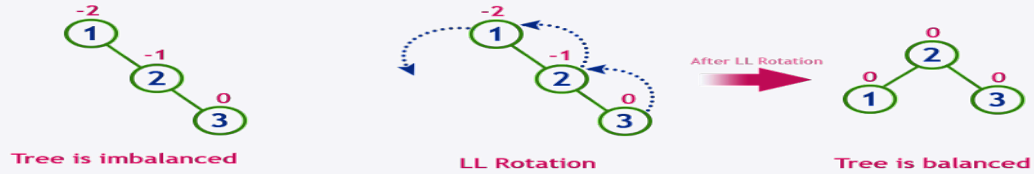
insert 1



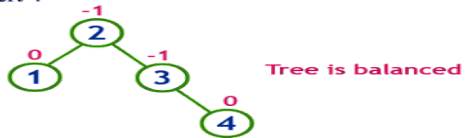
insert 2



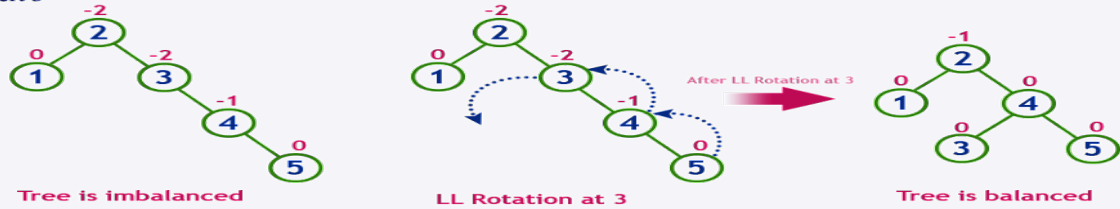
insert 3



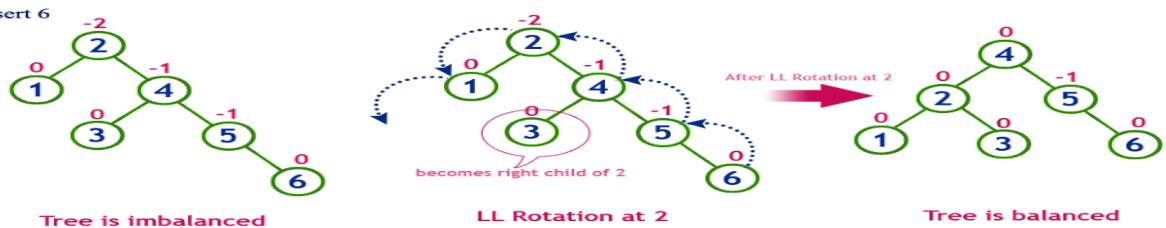
insert 4



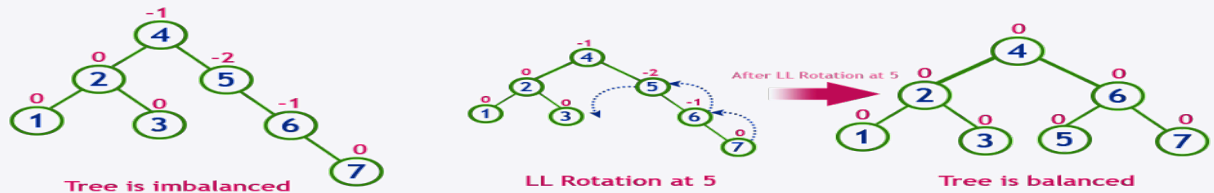
insert 5



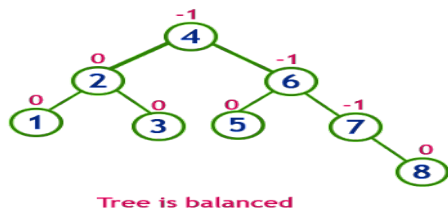
insert 6



insert 7



insert 8



Deletion in AVL Tree:-

The deletion of a node from an AVL tree is exactly the same as the deletion of a node from the binary search tree. After deletion of any particular node from AVL tree, the tree has to be restructured in order to preserve AVL property and thereby various rotations need to be applied

Algorithm

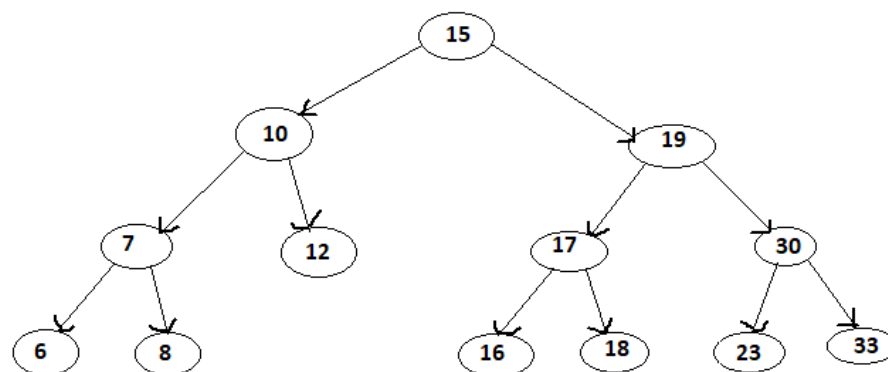
1. Initially, the AVL tree is searched to find the node to be deleted. The node to be deleted could be a leaf node, a node with one child or a node with two children.
2. (a) If the node to be deleted is a leaf node then simply make it NULL to remove.
(b) If the node to be deleted is not a leaf node then the node must be swapped with its inorder successor. Once the node is swapped, one can remove the node.
3. Check the balance factor of each node of the AVL tree after the deletion of the node.
4. Rebalance the AVL tree if the tree is unbalanced. For this, AVL rotations are used.

The deletion algorithm takes $O(\log n)$ time to delete a node.

Search in AVL Tree:-

As AVL tree is basically binary search tree, algorithm used for searching a node from binary search tree is used for searching a node from AVL tree.

An AVL tree can be searched for a key K by visiting nodes starting from root. The visited node is checked for the presence of K. If found, then the search stops otherwise if the value of K is less than the key value of visited node, then the left subtree is searched else the right subtree is searched.



Now to search the node 23 the steps are:

1. Compare 23 with root node 15. As $23 > 15$, move on
2. Compare 23 with node 19. As $23 > 19$, move on right
3. Compare 23 with node 30. As $23 < 30$, move on left
4. Compare 23 with node 23. As the match is found, declare the desired node 23 is present in the AVL tree.

B-TREE

A B-Tree is known as balanced sort or balanced m-way tree. It is used in external sorting. To reduce disk accesses, several conditions of the tree must be true.

- The height of the tree must be kept minimum.
- There must be no empty subtrees above the leaves of the tree.
- Leaves of the tree must all be at the same level.
- All nodes except the leaves must have at least some minimum of children.

Properties of B-Tree:-

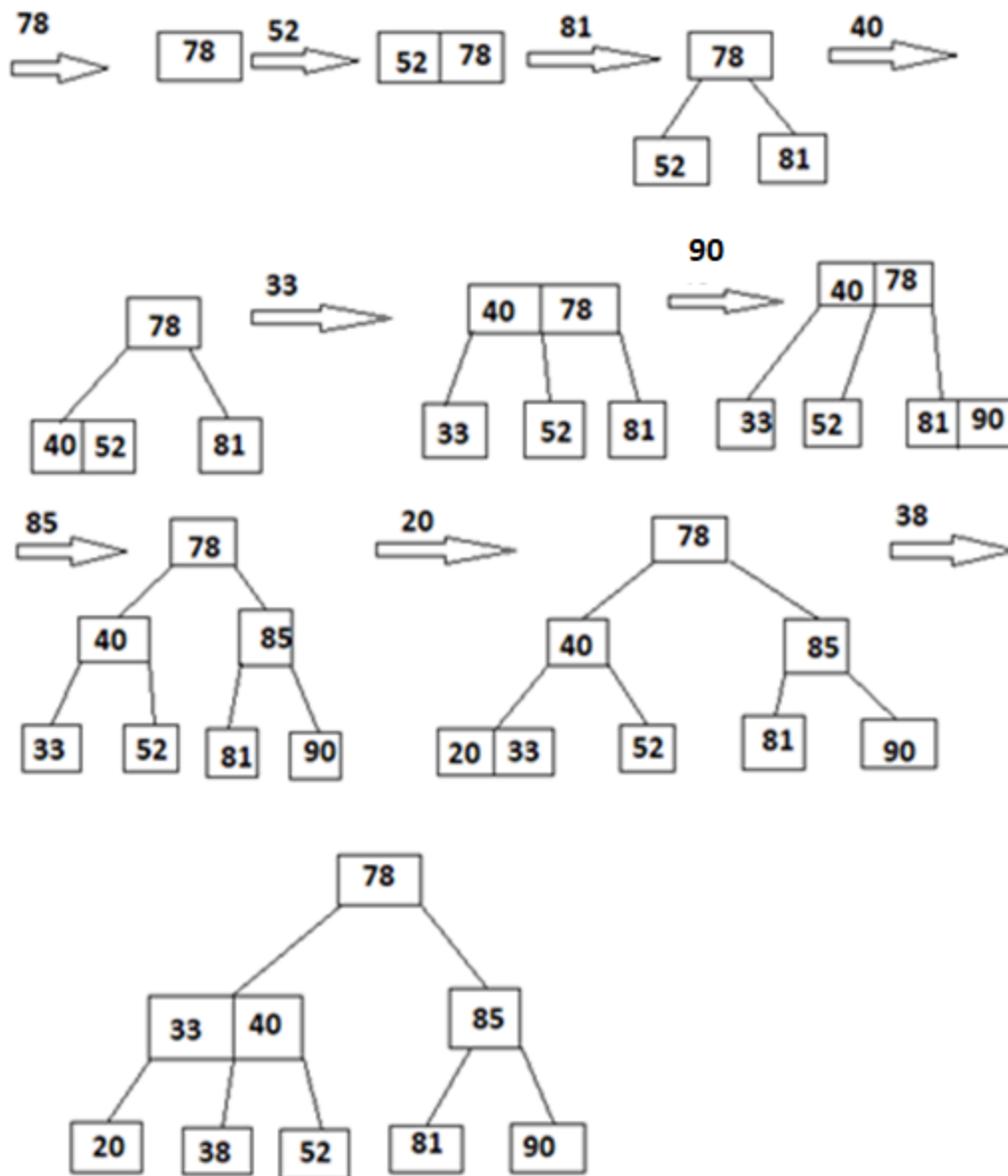
- Each node has a maximum of M children and minimum of $M/2$ children or any number from 2 to the maximum.
- Each node has one fewer keys than children with a maximum of (M-1) keys.
- Keys are arranged in a defined order within the node. All keys in the subtree to the left of a key are predecessor of the key and on the right are successors of the key.
- When a key is to be inserted into a full node, the node is splitted into two nodes, and the key with the median value is inserted in the parent node. In case the parent node is the root, then a new root is created.
- All leaves are on the same level i.e. there is no empty subtree above the level of the leaves.

Example

Construct a B tree of order 3 using the following elements:

78, 52, 81, 40, 33, 90, 85, 20, 38

Solution

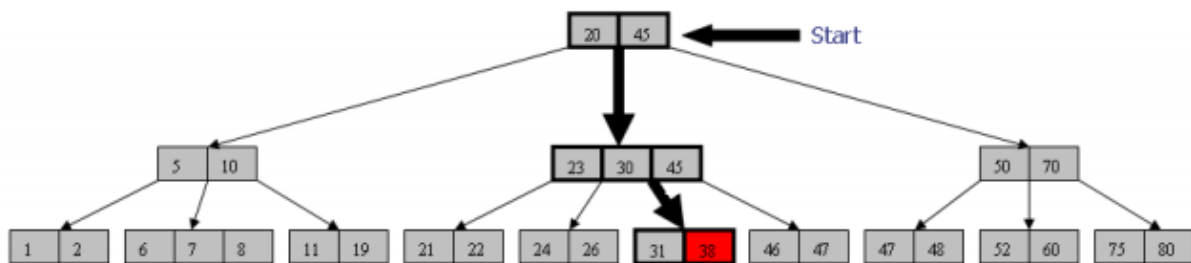


Search:-

Search is similar to the search in Binary Search Tree. Let the key to be searched be k . We start from the root and recursively traverse down. For every visited non-leaf node, if the node has the key, we simply return the node. Otherwise, we recur down to the appropriate child (The child which is just before the first greater key) of the node. If we reach a leaf node and don't find k in the leaf node, we return NULL.

Example:-

Searching 38 in B-Tree.



Traverse:-

Traversal is also similar to Inorder traversal of Binary Tree. We start from the leftmost child, recursively print the leftmost child, then repeat the same process for remaining children and keys. In the end, recursively print the rightmost child.

B-Tree Insertion:-

First search for the place where the new key should be inserted. If the node can accommodate the new key then insertion is simple.

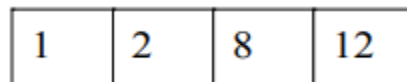
When a new key is to be inserted into a full node then

- The node is split into two nodes and the key with the median value is promoted to the parent node.
- The value greater than the median value are moved to a new node whereas the values less than middle value remains in the old node.

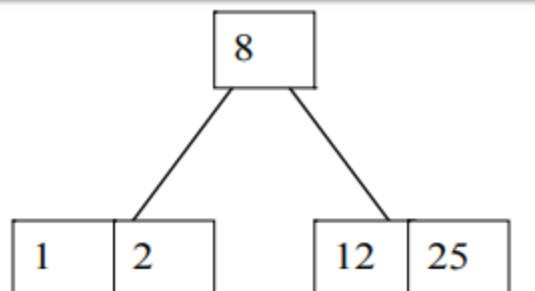
Example:-

Consider building a B-tree of degree 5 that is a balanced five way tree where each node can hold maximum four keys and have maximum five branches. Suppose it needs to contain the following values: 1 12 8 2 25 6 14 28 17 7 52 16 48 68 3 26 29 53 55 45

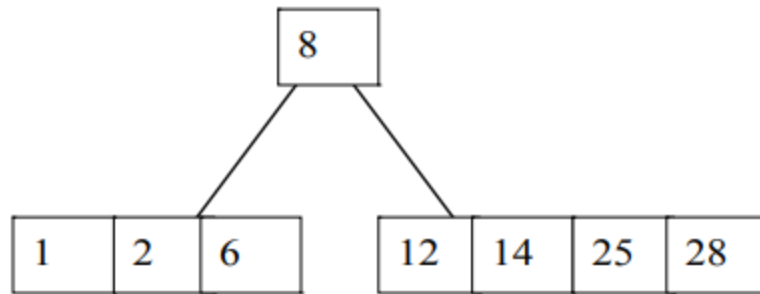
- The first four keys are inserted into one node. When they are inserted, they are sorted into the proper order.



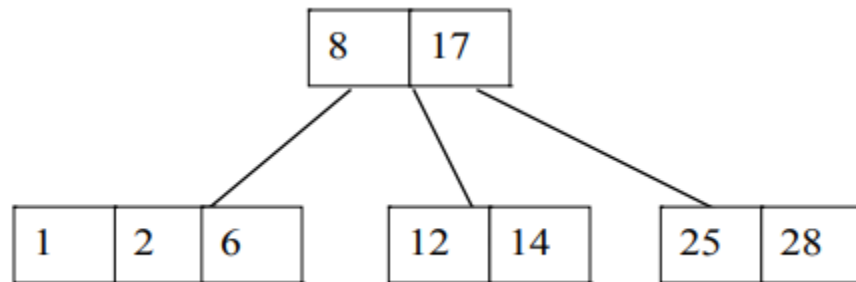
- When the fifth key 25 is to be added, this node is split into two and the median key 8 moves to a new node.



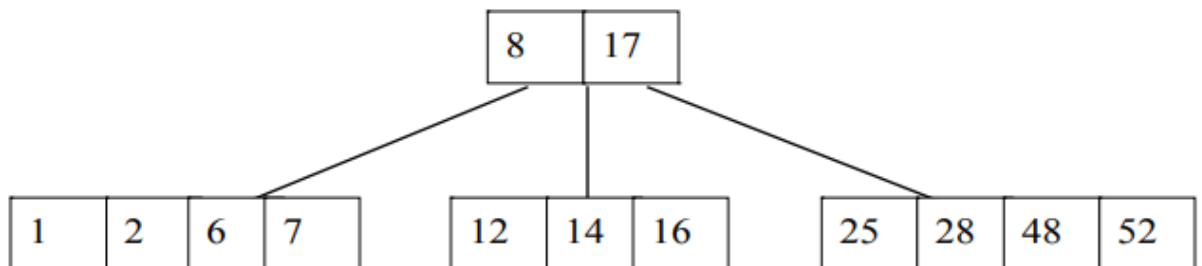
- 6, 14, 28 are inserted as:



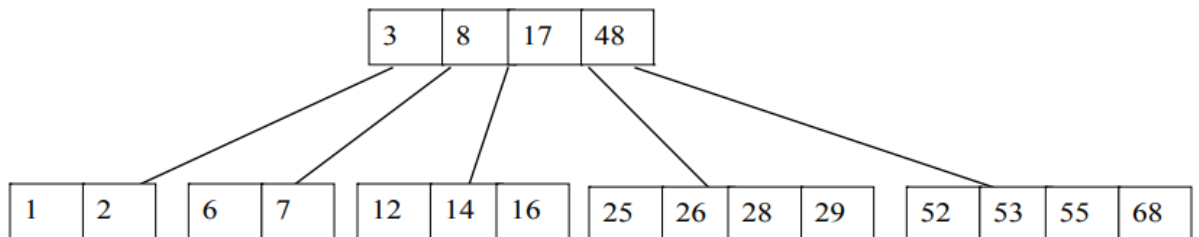
- 17 inserted, after inserted 17 the right leaf node would over-fill it, so we take the middle key and split the leaf.



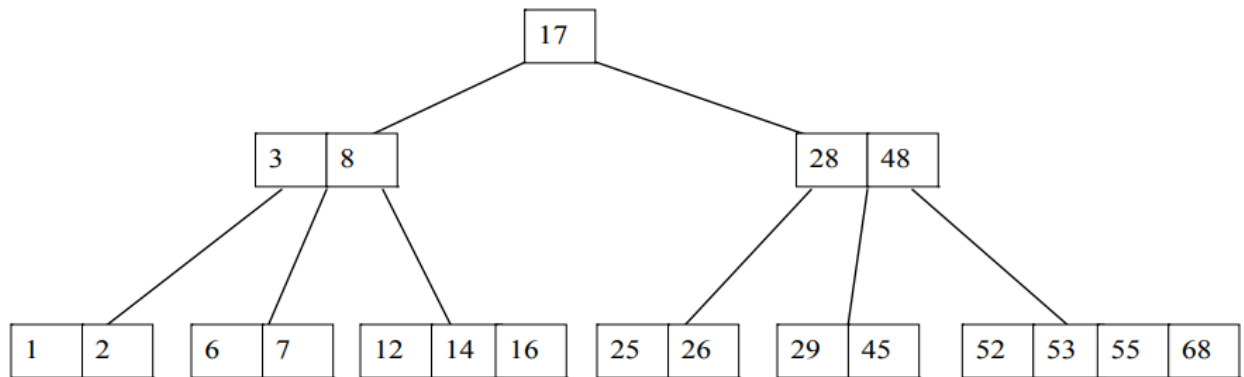
- 7, 52, 16, 48 are inserted as:



- Adding 68 causes us to split the right most leaf, promoting 48 to the root, and adding 3 causes us to split the left most leaf, promoting 3 to the root; 26, 29, 53, 55 then go into the leaves.



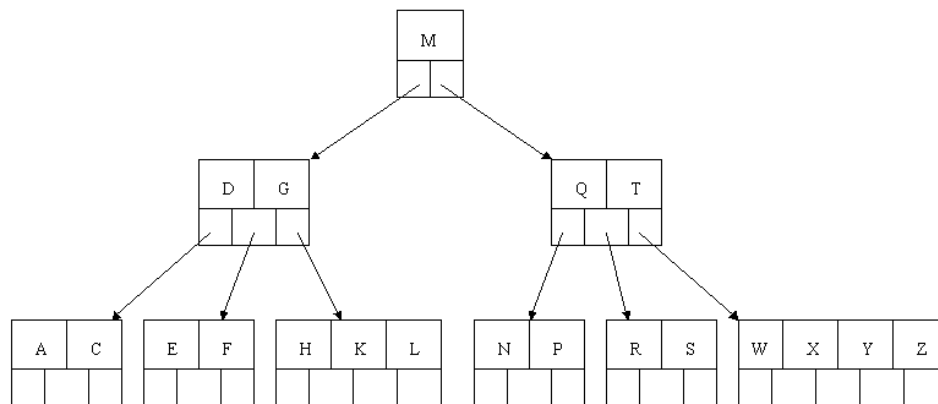
- Adding 45 causes a split of 25, 26, 28, 29 and promoting 28 to the root then causes the root to split.



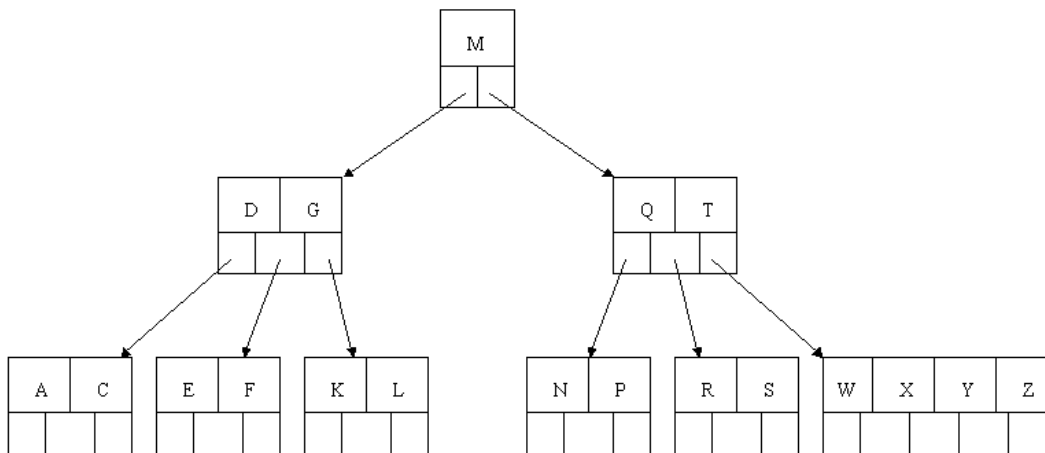
B-Tree Deletion:-

Deletion in a B Tree is similar to insertion. At first the node from which a value is to be deleted is searched. If found out, then the value is deleted. After deletion the tree is checked if it still follows B Tree properties.

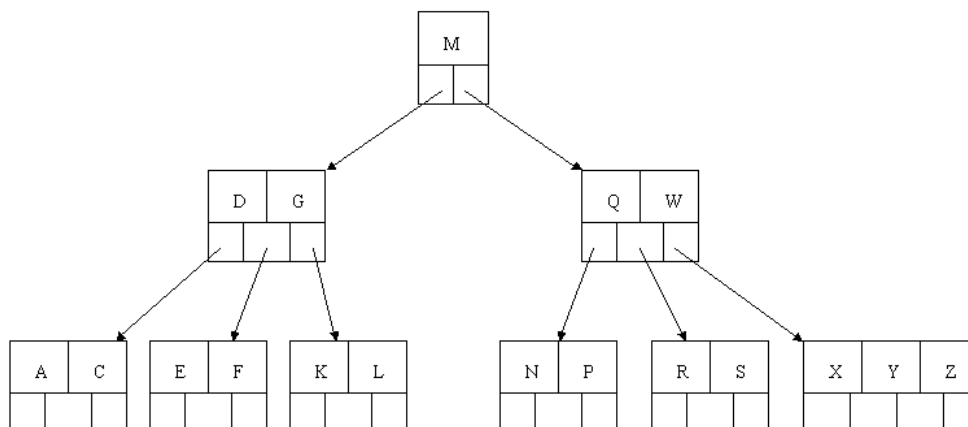
Let us take an example. The original B Tree taken is as follows:



Delete H. first it is found out. Since H is in a leaf and the leaf has more than the minimum number of keys, this is easy. We move the K over where the H had been and the L over where the K had been. This gives:



Next, delete the T. Since T is not in a leaf, we find its successor (the next item in ascending order), which happens to be W, and move W up to replace the T. That way, what we really have to do is to delete W from the leaf, which we already know how to do, since this leaf has extra keys. In ALL cases we reduce deletion to a deletion in a leaf, by using this method



B+ Tree:-

B+ Tree is an extension of B Tree which allows efficient insertion, deletion and search operations.

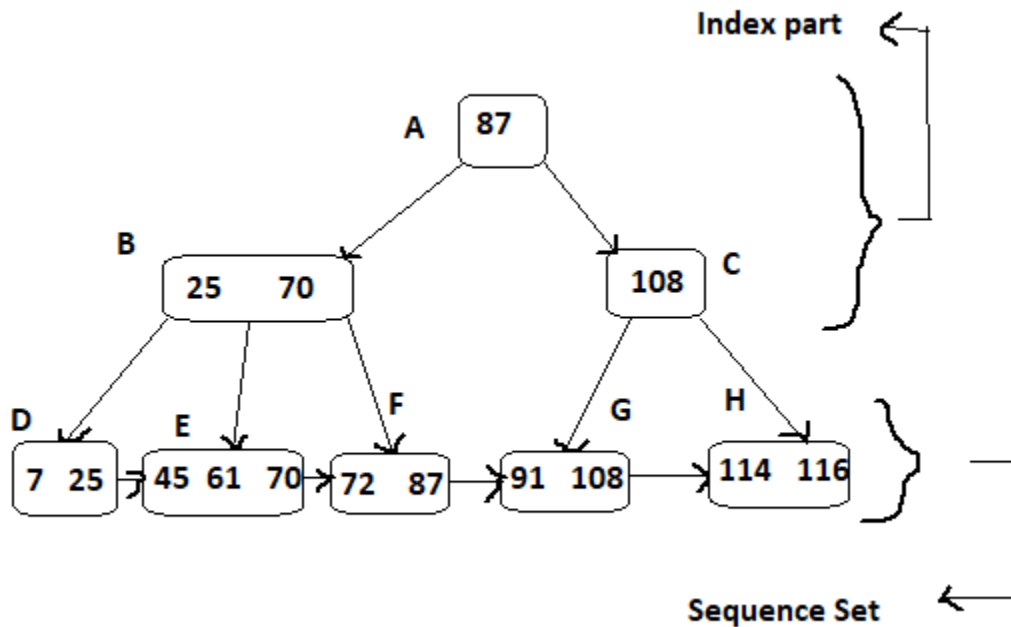
In B Tree, Keys and records both can be stored in the internal as well as leaf nodes. Whereas, in B+ tree, records (data) can only be stored on the leaf nodes while internal nodes can only store the key values.

The leaf nodes of a B+ tree are linked together in the form of a singly linked list to make the search queries more efficient.

Thus B+ Tree has two parts:

1. Index part

2. Sequence set



Advantages of B+ Tree

1. Records can be fetched in equal number of disk accesses.
2. Height of the tree remains balanced and less as compare to B tree.
3. We can access the data stored in a B+ tree sequentially as well as directly.
4. Keys are used for indexing.
5. Faster search queries as the data is stored only on the leaf nodes.

Insertion in B+ Tree

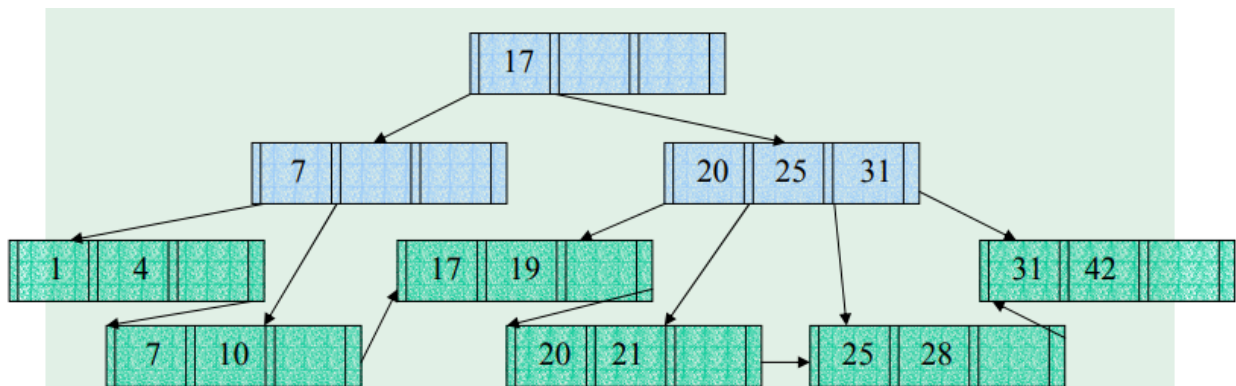
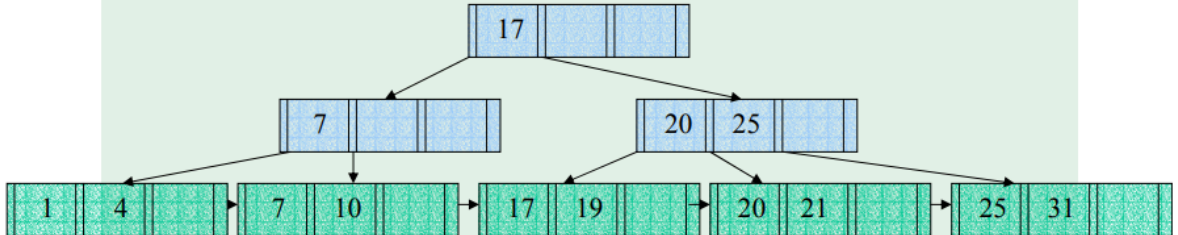
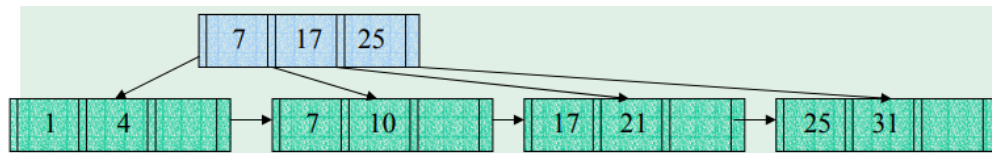
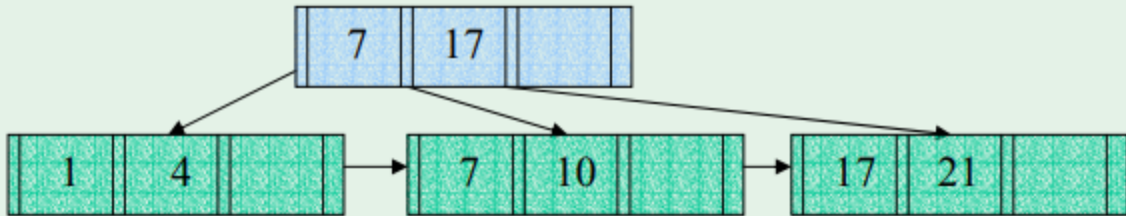
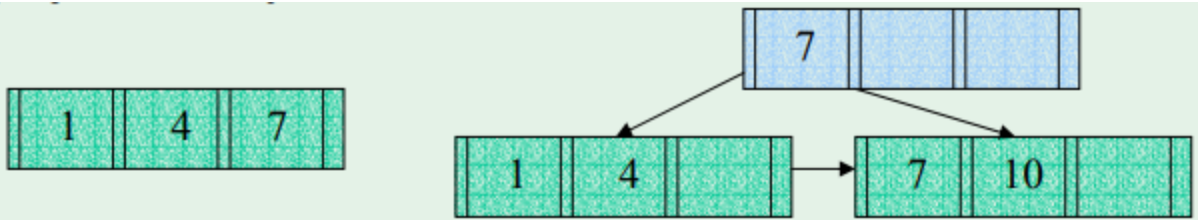
In order to insert a key into a B+ Tree, first a B+ tree search is performed to find the correct location for the key.

Step 1: Insert the new node as a leaf node

Step 2: If the leaf doesn't have required space, split the node and copy the middle node to the next index node.

Step 3: If the index node doesn't have required space, split the node and copy the middle element to the next index page.

Example: Construct a B+ tree for (1, 4, 7, 10, 17, 21, 31, 25, 19, 20, 28, 42) with n=4.



Deletion in B+ Tree:-

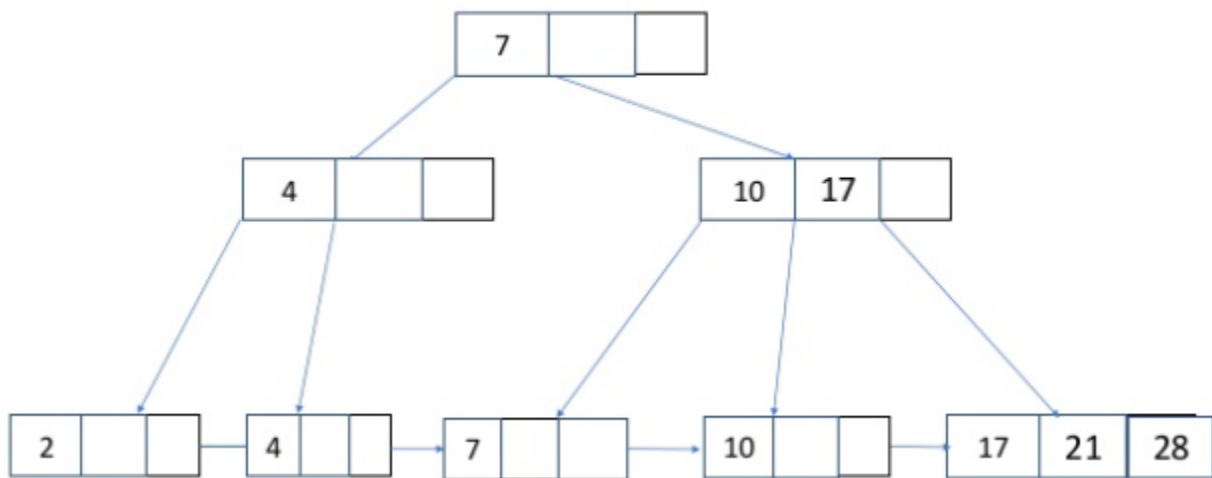
Step 1: Delete the key and data from the leaves.

Step 2: if the leaf node contains less than minimum number of elements, merge down the node with its sibling and delete the key in between them.

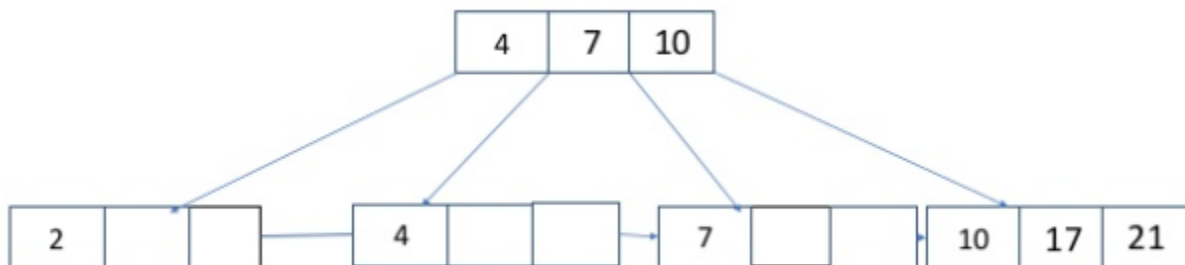
Step 3: if the index node contains less than minimum number of elements, merge the node with the sibling and move down the key in between them.

Example: -Create a B+ tree(2, 4, 7, 10, 17, 21, 28)

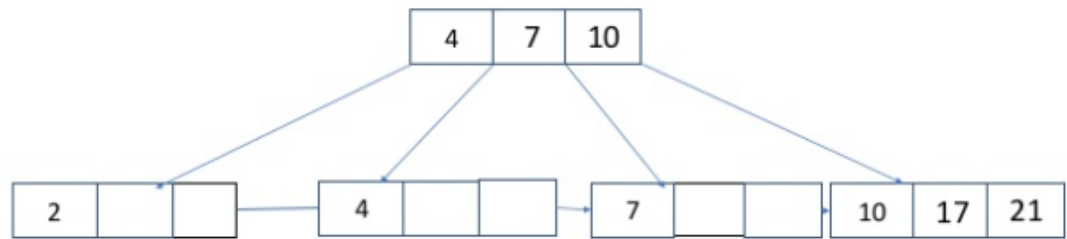
1. Deletion of 28



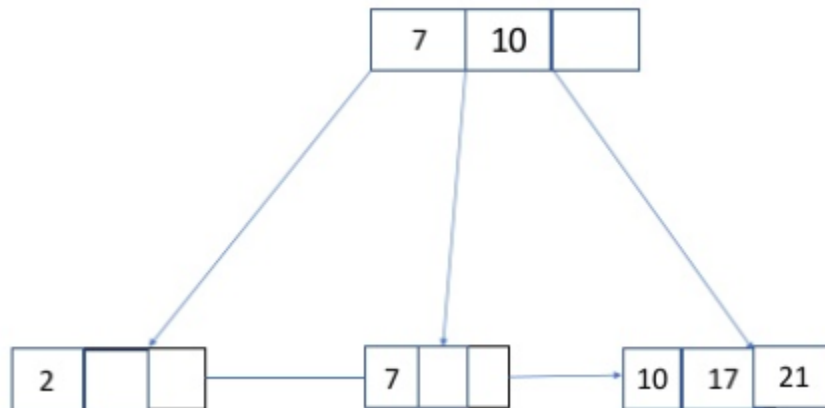
After deleted 28



2. Deletion of 4



After deleted 4



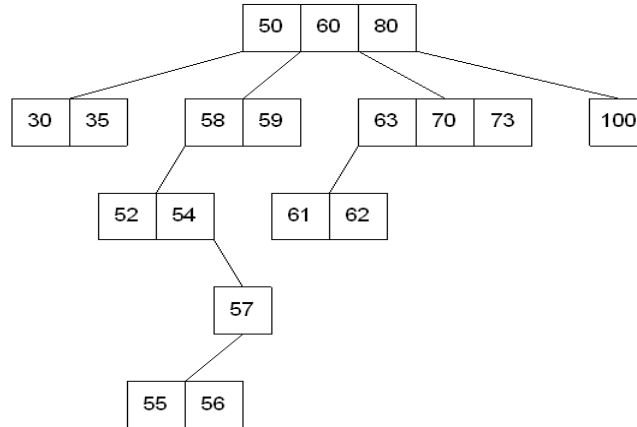
M – Way search tree:-

Trees having $(m - 1)$ keys and m children are called m -way search trees. Thus, an m -way search tree is a tree in which:

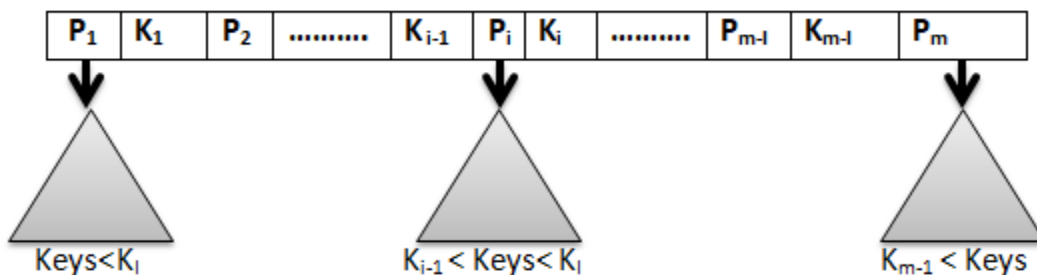
- The nodes hold between 1 to $m-1$ distinct keys.
- The keys in each node are sorted in ascending order.
- A node with k values has $k+1$ subtrees, where the subtrees may be empty.
- The keys in the first i children are smaller than the i th key.
- The keys in the last $m-i$ children are larger than the i th key.

M -way search tree is a generalization of binary search tree. A binary search tree is also called an m -way tree of order 2.

Example: - An m -way tree at order 4.



Each node has the following structure:-



Where: K_i are the keys and P_i are pointers to subtrees.

Advantages of m-way search tree

- They provide fast information retrieval.
- They provide fast updation.

Disadvantages of m-way search tree

- The tree is not balanced.
- Leaf nodes are on different levels.
- Bad space usage, tree can become skew.

Searching an m-way search tree

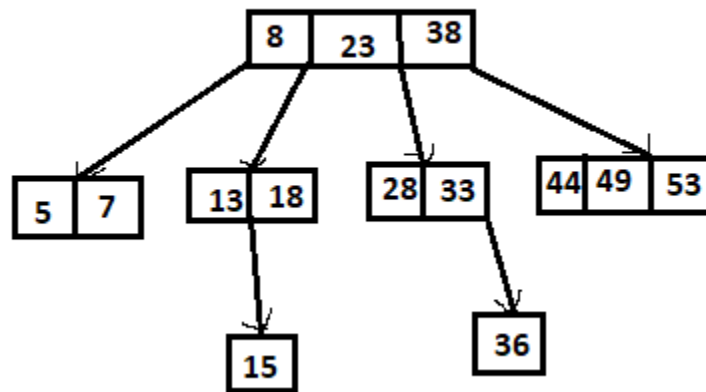
The key search from an m-way search tree is an extension of key search in a binary search tree. In m-way search tree, the key K is first searched sequentially from the key fields of the root node. If $K = K_j$, then the search is terminated returning the level at which the key is present. If $K < K_j$ for some j , then the subtree pointed by P_{j+1} is searched. This search continues till either the key element matches the search element K or a null pointer is encountered. Thus the procedure is:

1. If the key K is less than K_j , then the search is continued in m-way search tree pointed to by pointer P_{j+1} .
2. If the key K lies between K_j and K_{j+1} then the search is continued in m-way search tree pointed to by pointer P_j .

3. If the key K is greater than K_{m-1} then the search is continued in m -way search tree pointed to by pointer P_m

Example: Consider a 4-way search tree as shown below.

Suppose 15 is to be searched from this 4-way search tree. First the element is compared with the key values in the root. Since $8 < 15 < 23$, the search moves down to the subtree rooted at node [13, 18]. Since $13 < 15 < 18$, the search again moves down to the subtree rooted at [15]. Then the search terminates at node [15], since it matches with the search element returning the level at which the node is present.

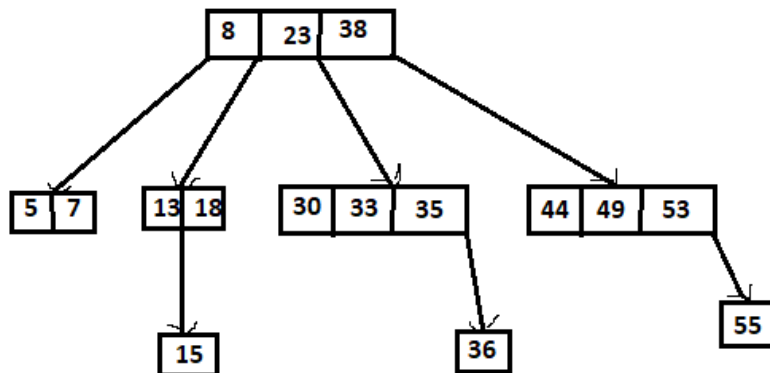


Inserting into an m-way search tree

The insertion of an element into an m -way search tree is performed by first finding the location of the node at which the element is to be inserted. If the node has already $m-1$ keys, then the element is inserted as a new node in the appropriate subtree of that node. Otherwise, the element can be inserted in the same node in its correct position.

Example: Consider a 4-way search tree. Suppose an element with key value 55 is to be inserted in this 4-way search tree. Use the search procedure to find the location of the node at which the element can be inserted and the location of the node found is [44, 49, 53].

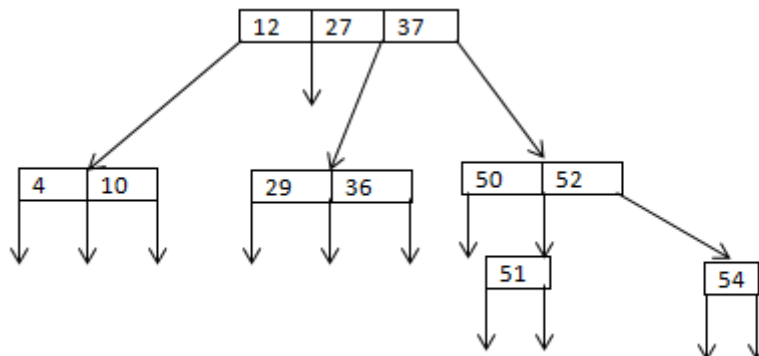
Since the node has already 3 keys, the element 55 is inserted as a new node in the subtree of [44, 49, 53]. Similarly, to insert key 33, the search falls at node [30, 35]. Since the tree is a 4-way search tree and can accommodate at most 3 keys, key 33 is inserted in [30, 35] and the node becomes as [30, 33, 35] as shown below:



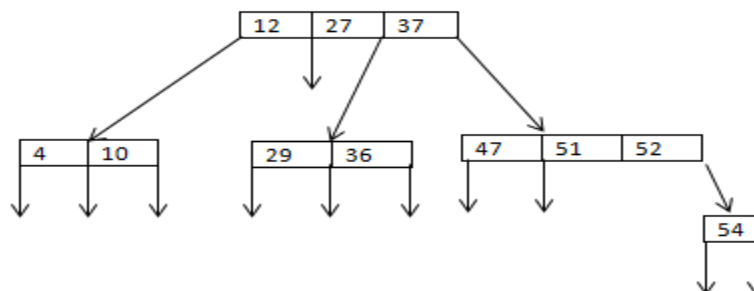
Deletion in m-way search tree

The deletion of an element from an m-way search tree is performed by first finding the location of the node at which the key is present.

Example:-

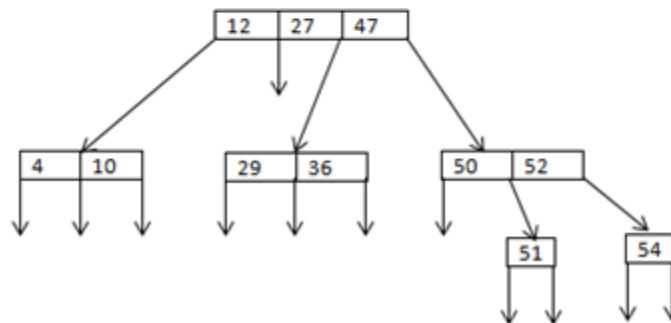
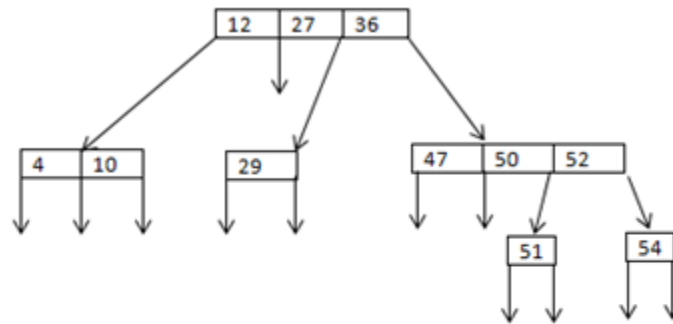


1. Delete 50 from the above tree



After deleting 50

2. Delete 37

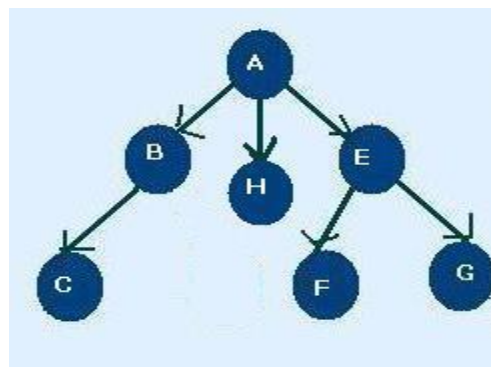


After deleting 37 two possible search trees

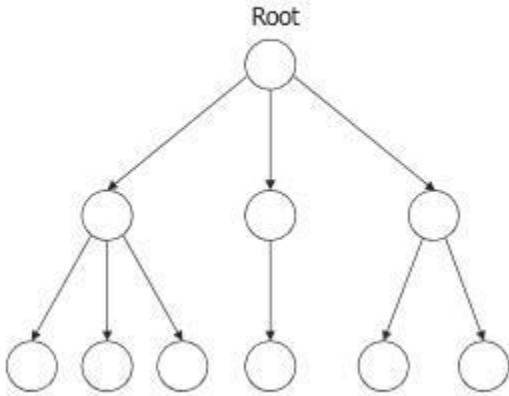
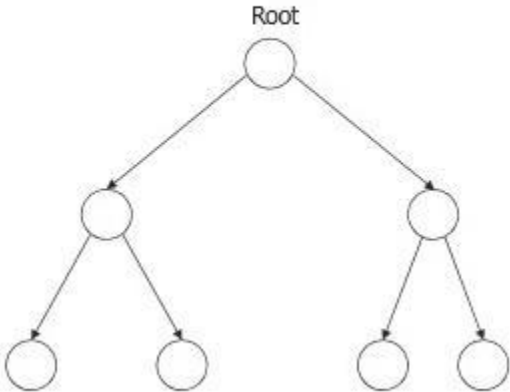
General Tree:-

General trees are those trees in which there could be any number of nodes that can be attached as a child node. The number of subtrees for each node may be varying.

Example: - The following tree is a general tree:



A binary tree is not a special case of a general tree. Binary trees and general trees are different objects. The main difference between binary tree and general tree:

General Tree	Binary Tree
A general tree is a data structure in that each node can have infinite number of children,	A Binary tree is a data structure in that each node has at most two nodes left and right.
A General tree can't be empty.	A Binary tree can be empty.
There is no limit on the degree of node in a general tree.	Nodes in a binary tree cannot have more than degree 2.
Subtrees of general tree are not ordered.	Subtrees of binary tree are ordered.
In general tree, root have in-degree 0 and maximum out-degree n.	In binary tree, root have in-degree 0 and maximum out-degree 2.
In general tree, each node have in-degree one and maximum out-degree n.	In binary tree, each node have in-degree one and maximum out-degree 2.
General tree 	Binary Tree 

Conversion of General Tree in to Binary Tree

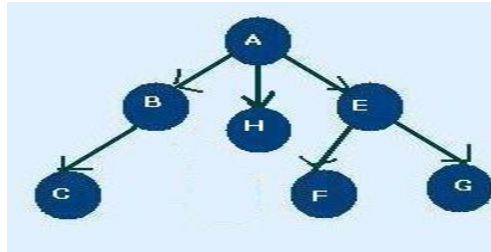
The process of converting general tree in to binary tree is given below:

1. Root node of general tree becomes root node of Binary Tree.
2. Now consider $T_1, T_2, T_3 \dots T_n$ are child nodes of the root node in general tree.
The left most child (T_1) of the root node in general tree becomes left most child

of root node in the binary tree. Now Node T2 becomes right child of Node T1, Node T3 becomes right child of Node T2 and so on in binary tree.

3. The same procedure of step 2 is repeated for each leftmost node in the general tree.

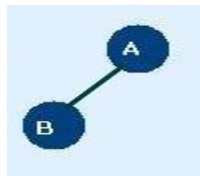
Consider following general tree:



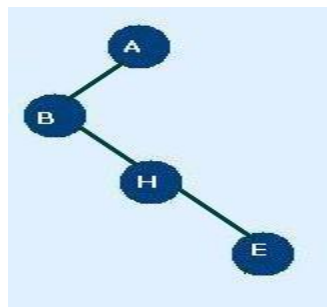
Step 1: Root Node of General tree becomes the Root node of binary tree.



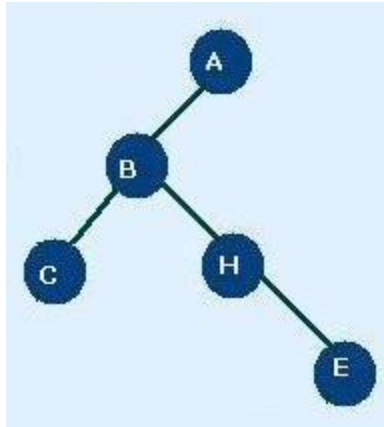
Step 2: Now Root Node (A) has three child Nodes (B, H, E) in general tree. The leftmost node (B) of the root node (A) in the general tree becomes the left most node of the root node (A) in binary tree.



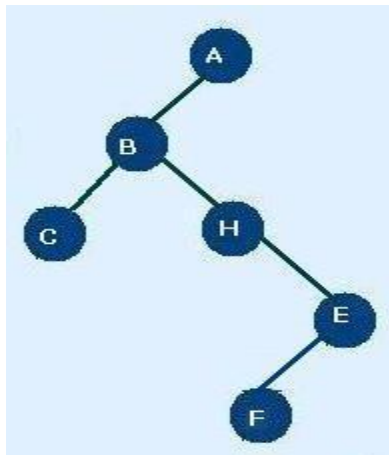
Step 3: Now Node H becomes the right node of B and Node E becomes the right node of H.



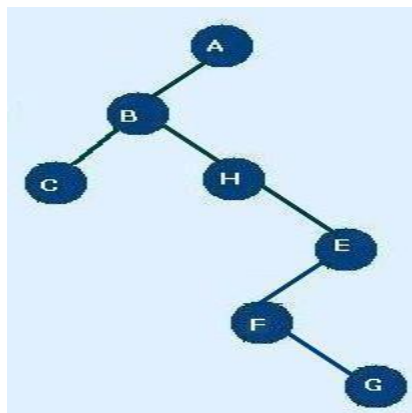
Step 4: Now Node B has only one left child node, which is C in general tree. So Node C becomes left child of Node B in binary tree.



Step 5: Now Node E has two child nodes (F, G). The leftmost node (F) of the node (E) in the general tree becomes the left most node of the node E in the binary tree.



Step 6: Now Node G becomes right node of Node F in binary tree.



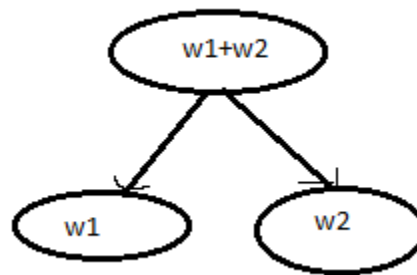
Huffman's Algorithm

Huffman's algorithm is a method for building an extended binary tree with a minimum weighted path length from a set of given weights. Initially construct a forest of singleton trees, one

associated with each weight. If there are at least two trees, choose the two trees with the least weight associated with their roots and replace them with a new tree, constructed by creating a root node whose weight is the sum of the weights of the roots of the two trees removed, and setting the two trees just removed as this new nodes children. This process is repeated until the forest consists of one tree.

The steps of algorithm are:-

1. Suppose there are 'n' weights $w_1, w_2, \dots, w_n$.
2. Take 2 minimum weights among the 'n' given inputs. Suppose w_1, w_2 are first two minimum, then the subtree will be:-



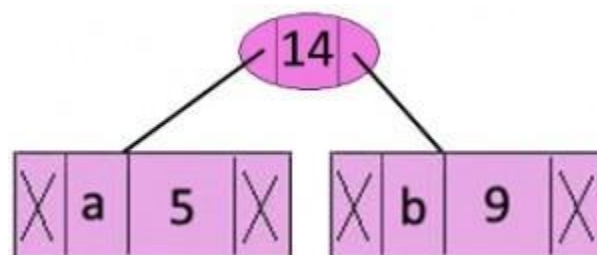
3. Now the remaining weights will be $w_3, w_1 + w_2, \dots, w_n$.
4. Create all subtree till the last weight. Thus Huffman algorithms construct the tree from the bottom up approach rather than top down approach.

Example of Huffman Algorithm

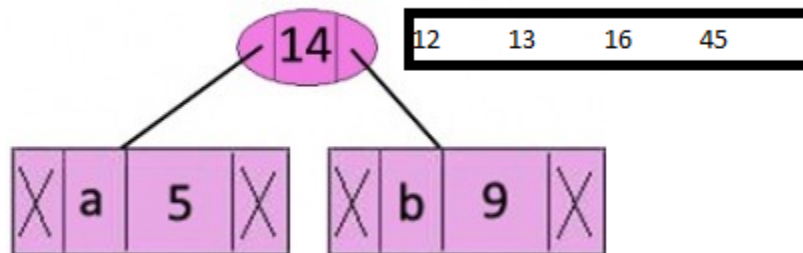
Suppose A, B, C, D, E, F are six elements with their weights as follows -

Item	A	B	C	D	E	F
Weights	5	9	12	13	16	45

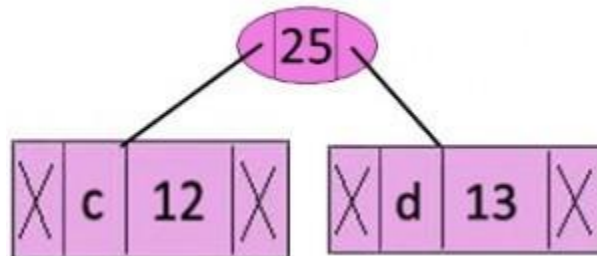
Step1:- Taking two nodes with minimum weight i.e. 5 and 9.



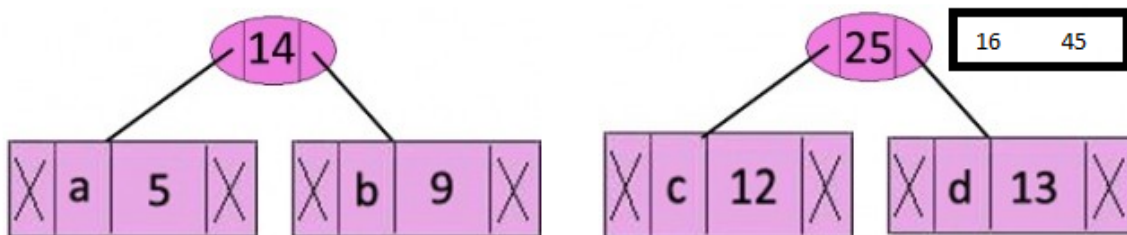
Now elements in the list are: 14, 12, 13, 16, 45.



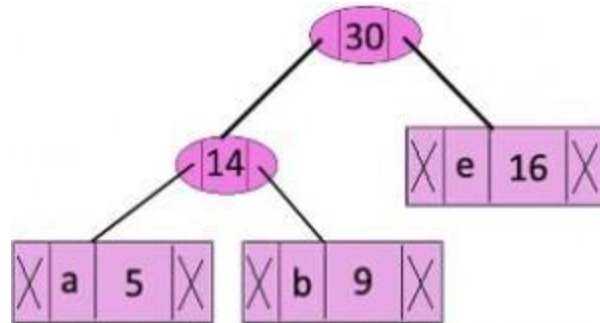
Step2:- Taking two nodes with minimum weight i.e. 12 and 13.



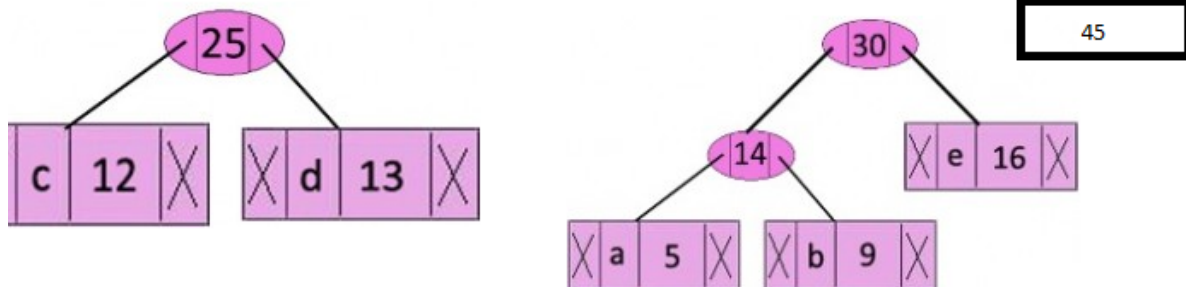
Now elements in the list are: 14, 25, 16, 45.



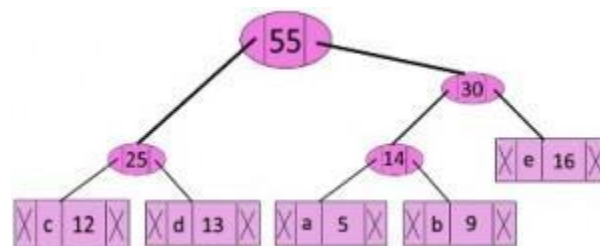
Step3:- Taking two nodes with minimum weight i.e. 16 and 14.



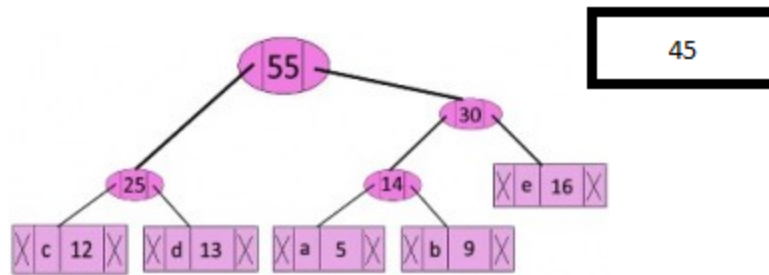
Now elements in the list are: 30, 25, 45.



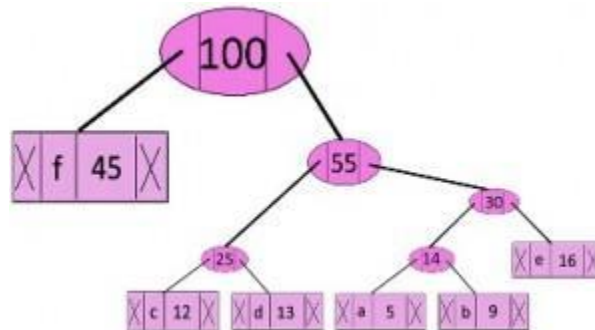
Step4:- Taking two nodes with minimum weight i.e. 25 and 30.



Now elements in the list are: 55, 45.

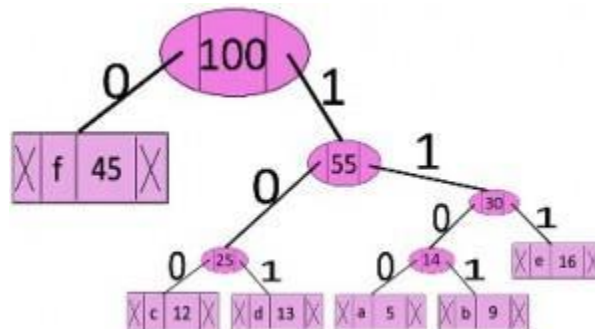


Step5:- Taking two nodes with minimum weights which are 55, 45.



Steps to print codes from Huffman Tree:

Traverse the tree formed starting from the root. Maintain an auxiliary array. While moving to the left child, write 0 to the array. While moving to the right child, write 1 to the array. Print the array when a leaf node is encountered.



The codes are as follows:

Symbol	Frequency	Code
F	45	0
C	12	100

D	13	101
A	5	1100
B	9	1101
E	16	111